

Sikuli basically automates anything you see on the screen of your desktop. It uses image recognition to identify and control GUI components. It comes up with basic text recognition OCR powered by Tesseract, which can be used to search for text in images.

So we can say that using Sikuli is WYSIWYS or What You See Is What You Script.

Sikuli can be used to automate testing through screenshots using Power Point slides while code lovers can use scripts in IDE to enhance its functionality. This framework is very useful in many scenarios like the following:

- It is best for use on Flash applications. For using the Selenium Web driver, we need the source code to develop the API. For example, if we need to automate the validation of Adobe Photoshop (whether an image got opened or not), then Sikuli can be very useful without using any API.
- It can be very useful in some scenarios where applications have a very complex source code yet very simple visualisation. So without going into the source code or some Xpath, we can automate and test the functionality of that application.
- It is very useful in cases in which the application code gets changed frequently but GUI components remain the same. In such cases, the functionality of an application can be validated using Sikuli.
- Sikuli can be very useful for game testers as well. Without using an API they can do some sort of testing on it. The system requirements for Sikuli are:
 - Windows XP and later, including Windows 8 and 10 (32-bit and 64-bit).
 - Linux/UNIX systems, depending on what prerequisites are available (32-bit or 64-bit).
 - Mac OSX 10.5 and later (64-bit only).

Installation of Sikuli on Windows

The path for downloading and setting up Sikuli is <https://launchpad.net/sikuli/+download>.

Download the Sikuli *setup.jar* file from this link.

Once the download is complete, click on the *sikulixsetup-1.1.1.jar* executable file and follow the instructions to install the Sikuli IDE.

Now, to run the Sikuli IDE, open command prompt, go to the path where you have just installed Sikuli and run *runsikulix.cmd*. This will open Sikulix IDE homepage.

Sikuli can be divided into two parts:

- Integrated Development Environment (IDE):** This is used to make scripts by taking screenshots.
- API/Sikuli script:** This part is used for GUI interaction of Jython and the Java library with keyboard or mouse events. Both these components are part of Sikulix.

Some basic features of Sikuli

Let us go through some basic functions of Sikuli.

Type (): The *type* command is a very basic command,

which we can use to enter input or text:

```
type ("This is Sample text example of type command")
```

The *type* command can also be used with a focused image, as while scripting we can focus on a particular area of application; then during execution, the *type ()* command will search that region first and type there. We can also use a modifier (as an option) with the *type* command to provide modifier keys as shown in the example below:

```
Type ("text", KeyModifier.ALT)
```

wait () and waitVanish () method: Both methods are used to slow down the script to wait for something or to make something vanish. They take an optional duration parameter, which can be a number of seconds, or the global parameter *FOREVER*, which will wait until something happens.

Find () and findAll (): These two are other common operations in Sikuli to search for things and interact with them. They are used when operating on a bunch of similar items on the screen. We can use some variable *r* to store the region as shown below:

```
r = find ( )
```

And later we can use that variable to call *wait ()*, *click ()*, *type ()*, and other functions so that it will restrict the search area and, hence, will help in speeding up the script. Selecting a region and assigning it to a variable also helps when there are multiple similar items on screen and we want to deal with a particular one at a time. For example:

```
r.click ( )
```

Highlight (): This is another basic command used to draw a box around a particular region.

Flow control technique in Sikuli

Sikuli uses some sort of control structure like a *FOR* loop with a combination of the *findAll ()* function. A sample coding of the *FOR* loop is given below:

```
--
Below_options= find (image1.png)
Checkboxes = below_options.findAll (image.png)
For checkbox in checkboxes:
    Checkbox.highlight(1)
--
```

Similarly, *if* and *while* flow control mechanisms can also be used, which allows you to do some more complex interactions through scripting.